รายงานย่อบทความ Publish Time Optimizations

ชื่อบทความ: Publish-Time Optimizations for Web-Based Visualizations
ผู้เขียน: Ron Pechuk และ Jeffrey Heer, University of Washington
งานประชุม: 2025 IEEE Visualization and Visual Analytics (VIS), pp. 226–230
DOI: 10.1109/VIS60296.2025.00051 | **IEEE Doc ID:** 11298742
ผู้จัดทำ: นายธีรภัทร ภู่ระย้า | **รหัสนิสิต:** 66362416
รายวิชา: 305432 เครือข่ายเซนเซอร์ (Assignment Paper IV)

บทสรุปบทความ

เว็บแสดงภาพข้อมูลแบบโต้ตอบต้องประมวลผลก่อนแสดงภาพและรับอินพุต ผู้วิจัยเสนอให้ย้ายงานที่ทำซ้ำข้ามผู้ใช้ไปยังช่วงเผยแพร่ผ่าน Mosaic Publisher ซึ่งเตรียมข้อมูล ผลคิวรี และภาพเริ่มต้นล่วงหน้า การทดลองกับภาพข้อมูล 5 รูปแบบพบว่าชุดกลยุทธ์รวมลดเวลาเห็นภาพเฉลี่ย 83.7% และเวลาพร้อมโต้ตอบ 33.3% แลกกับพื้นที่ทรัพยากรเพิ่ม 21.2% งานนี้แสดงประโยชน์ของการเตรียมงานก่อนเปิดหน้า แต่ยังเหลือต้นทุน Hydration ฝั่งไคลเอนต์ [1]

ที่มาและปัญหาวิจัย

การสร้างแคชตอนใช้งานช่วยคิวรีครั้งต่อไป แต่ผู้ใช้ครั้งแรกยังต้องรอโหลดข้อมูล คำนวณคิวรี และวาดภาพ ผู้วิจัยจึงแยก Time-to-Render (TTR) ซึ่งนับจากร้องขอหน้าจนเห็น visualization ครบ ออกจาก Time-to-Activation (TTA) ซึ่งนับจนระบบพร้อมรับการกรองหรือเลือกข้อมูล พร้อมวัด Storage Cost (SC) ของทรัพยากรเผยแพร่ การแยกสองเวลานี้ทำให้ตรวจได้ว่าภาพที่มาเร็วขึ้นพร้อมใช้งานจริงหรือยัง ต่างจากการนับเพียงเนื้อหาใด ๆ ปรากฏบนหน้า (Section 3.1)

วิธีดำเนินการและสถาปัตยกรรม

Mosaic ใช้ coordinator กลางจัดการคิวรีของ views และเชื่อมการโต้ตอบ Publisher อ่านข้อกำหนด YAML เป็น Abstract Syntax Tree (AST) แล้วเลือกเตรียมงานตามระดับการปรับปรุง DuckDB แบบ in-process ใช้คำนวณตอนเผยแพร่ ส่วน runtime รองรับฐานข้อมูลในเบราว์เซอร์หรือบนเซิร์ฟเวอร์ กระบวนการหลักมีดังนี้ (Sections 4.1–4.3)

จำลองการโต้ตอบ: กระตุ้นตัวรับอินพุตบน virtual DOM เพื่อค้นคิวรีที่เกิดจากการกรองและการเลือกข้อมูล

เก็บผลคำนวณ: รันงานเตรียมข้อมูล คิวรีเริ่มต้น และ pre-aggregations แล้วบันทึกเป็นทรัพยากรที่นำไปเผยแพร่ได้

ปรับข้อกำหนด: rewrite AST ให้ส่วนเตรียมข้อมูลชี้ผลที่คำนวณไว้ ส่วน query cache และ materialized views โหลดเข้าตัวกลางและฐานข้อมูลได้โดยตรง

เตรียมภาพและเริ่มระบบ: view ที่ไม่โต้ตอบส่งเฉพาะภาพที่เรนเดอร์ไว้ ส่วน interactive view ส่งภาพเริ่มต้นพร้อม JavaScript สำหรับเติมแคชและเริ่มคอมโพเนนต์โต้ตอบ

Hydration ปัจจุบันยังสร้างคอมโพเนนต์ผ่าน client-side rendering ตามปกติ ผู้วิจัยยังไม่ได้ทดลอง progressive hydration และเสนอการทำ Hydration ที่ละเอียดขึ้นเป็นงานอนาคต การมีภาพล่วงหน้าจึงไม่ได้ทำให้งานเปิดระบบโต้ตอบหายไป

การทดลองและผลลัพธ์

ผู้วิจัยทดสอบ airlines, property, flights, gaia และ taxis ที่ข้อมูล 10k–10M แถว บน MacBook Pro ปี 2021, M1 Pro, RAM 16 GB, macOS 15.2 และ Node.js 22.9.0 ใช้ local server กับ DuckDB-WASM เปรียบเทียบ none ที่คำนวณตอน runtime กับระดับปรับปรุงที่เพิ่มสะสมจากเตรียมข้อมูล เพิ่มแคช แล้วเพิ่มภาพเริ่มต้น (Section 5)

ตารางที่ 1 ค่าเฉลี่ยการเปลี่ยนแปลงเทียบ none ข้ามรูปแบบภาพและขนาดข้อมูล พร้อมส่วนเบี่ยงเบนมาตรฐาน **σ** (ต้นฉบับ Table 1, หน้า PDF 4)

ระดับปรับปรุง	ΔSC (%)	ΔTTR (%)	ΔTTA (%)
+prep	0.0 ± 0.0	−0.6 ± 4.0	−1.0 ± 2.5
+cache	+21.1 ± 29.8	−16.6 ± 1.3	−32.3 ± 8.9
+most (ชุดรวม)	+21.2 ± 29.9	−83.7 ± 2.1	−33.3 ± 9.6

+prep ช่วยน้อยเพราะชุดข้อมูลไม่มีงานแปลงหรือคอลัมน์ส่วนเกิน +cache ลด TTA ได้มาก โดยเฉพาะข้อมูลตั้งแต่ 1M แถว ส่วนการเพิ่มภาพเริ่มต้นทำให้ TTR ลดชัด แต่ลด TTA เพิ่มเพียง 1.0 จุดเปอร์เซ็นต์ เมื่อเทียบค่าเฉลี่ยการลดจาก baseline เดียวกัน (คำนวณจาก 33.3 − 32.3) Figure 4 อธิบายว่าต้นทุน Hydration ยังเกือบเท่าการเรนเดอร์ฝั่งไคลเอนต์เต็มชุด ทั้งนี้ Table 1 เรียกชุดรวมว่า +most ส่วนกราฟและข้อความใช้ +prerender

คุณูปการของงานวิจัย

บทความผสานกรอบการประเมินที่วัด TTR, TTA และ SC ร่วมกัน เข้ากับเครื่องมือเผยแพร่ที่จำลองอินพุต เก็บผลคิวรี และ rewrite AST หลักฐานเชิงระบบแสดงว่าแคชกับภาพล่วงหน้าช่วยคนละช่วง พร้อมแสดงต้นทุน Hydration ที่ยังเหลือ แม้ว่าภาพเริ่มต้นจะปรากฏเร็วขึ้นแล้ว

จุดเด่นและข้อจำกัด

จุดเด่นคือมี baseline และทดสอบหลายรูปแบบภาพ ทำให้เห็นผลของการเพิ่มกลยุทธ์เป็นขั้น อย่างไรก็ตาม SC เพิ่ม 21.2% ± 29.9 จึงแตกต่างมากระหว่างเงื่อนไข ทรัพยากรต้องสร้างใหม่เมื่อข้อมูลเปลี่ยน และการเก็บหลายสถานะอาจกินพื้นที่สูง การทดลองบน laptop เครื่องเดียวและ local server ยังไม่ครอบคลุมมือถือหรือเน็ตช้า อีกทั้งบทความไม่แจกแจงจำนวนรอบทำซ้ำ จึงยังตรวจความแปรปรวนระดับการรันไม่ได้

ข้อสรุปและแนวทางที่ผู้วิจัยเสนอ

จากผลทดลอง แคชลดงานก่อนเปิดการโต้ตอบ ส่วนภาพล่วงหน้าลดเวลารอเห็น visualization แต่ยังเหลือต้นทุน Hydration เวลาเห็นภาพกับเวลาพร้อมโต้ตอบจึงลดลงไม่เท่ากัน ผลด้านความเร็วเกิดพร้อมพื้นที่ที่เพิ่มขึ้นและอยู่ในขอบเขตสภาพทดลอง ผู้วิจัยเสนอให้ศึกษาวิธี hydration ที่ละเอียดขึ้น กลยุทธ์ผสม publish/runtime และ incremental regeneration รวมถึงการเลือกชุดปรับปรุงโดยอัตโนมัติจากข้อมูลและข้อกำหนดภาพ แนวทางเหล่านี้ยังเป็นงานอนาคต (Sections 5–6)

เอกสารอ้างอิง

[1] R. Pechuk and J. Heer, “Publish-Time Optimizations for Web-Based Visualizations,” in 2025 IEEE VIS, 2025, pp. 226–230, doi: 10.1109/VIS60296.2025.00051.